

# Oracle HR Schema – Subqueries Analytical Reports

**Name:** Al-Jolanda Al-Handhali

## Contents

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| <b>1. Salary Analysis Using Subqueries</b>                              | 3  |
| 1.1 Employees earning more than the company average salary              | 3  |
| 1.2 Employees earning less than the company's highest salary            | 3  |
| 1.3 Departments whose average salary exceeds company average salary     | 4  |
| 1.4 Employees earning the highest salary within their department        | 4  |
| <b>2. Department &amp; Location Analysis</b>                            | 5  |
| 2.1 Employees working in departments located in the United States       | 5  |
| 2.2 Departments that currently contain employees                        | 5  |
| 2.3 Departments that currently have no employees                        | 6  |
| 2.4 Employees working in departments located in Europe region           | 6  |
| <b>3. Manager &amp; Hierarchy Analysis</b>                              | 7  |
| 3.1 Employees earning more than their manager                           | 7  |
| 3.2 Managers supervising at least one employee earning more than 10,000 | 7  |
| 3.3 Employees whose manager works in the same department                | 8  |
| 3.4 Employees whose manager works in a different department             | 8  |
| <b>4. Inline View &amp; Query Execution</b>                             | 9  |
| 4.1 Employees earning more than 10,000                                  | 9  |
| 4.2 Departments with average salary greater than 9,000                  | 9  |
| 4.3 Departments with more than five employees                           | 10 |
| <b>5. Advanced Subquery Logic</b>                                       | 10 |
| 5.1 Employees earning above their department average salary             | 10 |
| 5.2 Employees earning above average salary for their job role           | 11 |
| 5.3 Departments where all employees earn more than 5,000                | 11 |
| 5.4 Employees who are the only employees in their department            | 12 |
| <b>6. Executive HR Analytics Challenge</b>                              | 12 |
| 6.1 Employees earning above company average salary                      | 12 |
| 6.2 Departments with more employees than Department 50                  | 13 |
| 6.3 Employees working in the department with the highest average salary | 13 |
| 6.4 Employees earning the second-highest salary                         | 14 |
| 6.5 Managers whose employees' average salary exceeds 10,000             | 14 |

## 1. Alary Analysis Using Subqueries

### 1.1 Employees earning more than the company average salary

#### Business Requirement:

- Find employees whose salary is above the overall company average.

#### SQL:

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary)
                FROM employees);
```

#### Subquery Type:

- Scalar Subquery
- Non-Correlated Subquery

#### Execution Logic:

1. Oracle calculates company average salary.
2. The value is returned as a single number.
3. Outer query compares each employee's salary against that value.

### 1.2 Employees earning less than the company's highest salary

#### Business Requirement:

- Exclude employees receiving the maximum salary.

#### SQL:

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary < (SELECT MAX(salary)
                FROM employees);
```

#### Subquery Type:

- Scalar Subquery
- Non-Correlated

#### Execution Logic:

1. Subquery finds highest salary.
2. Outer query returns employees earning less than that amount.

### 1.3 Departments whose average salary exceeds company average salary

#### Business Requirement:

- Identify departments performing above the company salary benchmark.

#### SQL:

```
SELECT department_id
FROM employees
GROUP BY department_id
HAVING AVG(salary) > (SELECT AVG(salary)
                      FROM employees);
```

#### Subquery Type:

- Scalar Subquery

#### Execution Logic:

1. Company average salary calculated once.
2. Each department average is calculated.
3. Departments exceeding company average are displayed.

### 1.4 Employees earning the highest salary within their department

Write here

#### Business Requirement:

- Find top earners in every department.

#### SQL:

```
SELECT employee_id, first_name, last_name, department_id, salary
FROM employees e
WHERE salary = (SELECT MAX(salary)
                FROM employees
                WHERE department_id = e.department_id);
```

#### Subquery Type:

- Correlated Subquery

#### Execution Logic:

1. Oracle reads one employee.
2. Subquery finds maximum salary in that employee's department.
3. Employees are returned if salaries match.

## 2. Department & Location Analysis

### 2.1 Employees working in departments located in the United States

#### Business Requirement:

- List of employees assigned to US-based departments.

#### SQL:

```
SELECT employee_id, first_name, last_name
FROM employees
WHERE department_id IN
    (SELECT department_id
     FROM departments
     WHERE location_id IN
         (SELECT location_id
          FROM locations
          WHERE country_id = 'US'));
```

#### Subquery Type:

- Nested Multi-row Subquery

#### Execution Logic:

- Locations → Departments → Employees

### 2.2 Departments that currently contain employees

#### Business Requirement:

- Identify active departments.

#### SQL:

```
SELECT department_id, department_name
FROM departments
WHERE department_id IN
    (SELECT DISTINCT department_id
     FROM employees
     WHERE department_id IS NOT NULL);
```

#### Subquery Type:

- Multi-row Subquery

#### Execution Logic:

- Subquery returns all department IDs appearing in Employees.

## 2.3 Departments that currently have no employees

### Business Requirement:

- Identify unused departments.

### SQL:

```
SELECT department_id, department_name
FROM departments
WHERE department_id NOT IN
    (SELECT department_id
     FROM employees
     WHERE department_id IS NOT NULL);
```

### Subquery Type:

- Multi-row Subquery

### Execution Logic:

- Returns departments absent from Employees.

## 2.4 Employees working in departments located in Europe region

### Business Requirement:

- List of employees assigned to departments located in Europe.

### SQL:

```
SELECT employee_id, first_name, last_name
FROM employees
WHERE department_id IN
    (
        SELECT department_id
        FROM departments
        WHERE location_id IN
            (
                SELECT location_id
                FROM locations
                WHERE country_id IN
                    (
                        SELECT country_id
                        FROM countries
                        WHERE region_id =
                            (
                                SELECT region_id
                                FROM regions
                                WHERE region_name = 'Europe'
                            )
                    )
            )
    )
```

### Subquery Type:

- Nested Subquery

### Execution Logic:

- Region → Countries → Locations → Departments → Employees

### 3. Manager & Hierarchy Analysis

#### 3.1 Employees earning more than their manager

**Business Requirement:**

- Identify employees paid more than their direct manager.

**SQL:**

```
SELECT employee_id, first_name, last_name, salary
FROM employees e
WHERE salary >
    (
        SELECT salary
        FROM employees
        WHERE employee_id = e.manager_id
    );
```

**Subquery Type:**

- Correlated Scalar Subquery

**Execution Logic:**

- Manager salary retrieved for each employee and compared.

#### 3.2 Managers supervising at least one employee earning more than 10,000

**Business Requirement:**

- Find managers overseeing highly paid employees.

**SQL:**

```
SELECT employee_id, first_name, last_name
FROM employees
WHERE employee_id IN
    (
        SELECT DISTINCT manager_id
        FROM employees
        WHERE salary > 10000
    );
```

**Subquery Type:**

- Multi-row Subquery

**Execution Logic:**

- Subquery returns managers of employees earning above 10,000.

### 3.3 Employees whose manager works in the same department

#### Business Requirement:

- Find employees sharing department with manager.

#### SQL:

```
SELECT employee_id, first_name, last_name
FROM employees e
WHERE department_id =
    (
        SELECT department_id
        FROM employees
        WHERE employee_id = e.manager_id
    );
```

#### Subquery Type:

- Correlated Subquery

#### Execution Logic:

- Compare employee department with manager department.

### 3.4 Employees whose manager works in a different department

#### Business Requirement:

- Find cross-department reporting relationships.

#### SQL:

```
SELECT employee_id, first_name, last_name
FROM employees e
WHERE department_id <>
    (
        SELECT department_id
        FROM employees
        WHERE employee_id = e.manager_id
    );
```

#### Subquery Type:

- Correlated Subquery

#### Execution Logic:

- Compare employee and manager departments.



## 4. Inline View & Query Execution

### 4.1 Employees earning more than 10,000

**SQL:**

```
SELECT *  
FROM  
(  
    SELECT employee_id,  
           first_name,  
           last_name,  
           salary  
    FROM employees  
) emp_view  
WHERE salary > 10000;
```

**Subquery Type:**

- Inline View

**Execution Logic:**

- Inner query creates temporary result set, outer query filters it.

### 4.2 Departments with average salary greater than 9,000

**SQL:**

```
SELECT *  
FROM  
(  
    SELECT department_id,  
           AVG(salary) avg_salary  
    FROM employees  
    GROUP BY department_id  
) dept_avg  
WHERE avg_salary > 9000;
```

**Subquery Type:**

- Inline View

**Execution Logic:**

- Department averages calculated first, then filtered.

### 4.3 Departments with more than five employees

**SQL:**

```
SELECT *  
FROM  
(  
    SELECT department_id,  
           COUNT(*) emp_count  
    FROM employees  
    GROUP BY department_id  
) dept_count  
WHERE emp_count > 5;
```

**Subquery Type:**

- Inline View

**Execution Logic:**

- Employee counts calculated then filtered.

## 5. Advanced Subquery Logic

### 5.1 Employees earning above their department average salary

**Business Requirement:**

- Identify above-average performers within departments.

**SQL:**

```
SELECT employee_id, first_name, last_name, department_id, salary  
FROM employees e  
WHERE salary >  
    (  
        SELECT AVG(salary)  
        FROM employees  
        WHERE department_id = e.department_id  
    );
```

**Subquery Type:**

- Correlated Subquery

**Execution Logic:**

- Department average recalculated for each employee.

## 5.2 Employees earning above average salary for their job role

### Business Requirement:

- Compare employees with peers in same job.

### SQL:

```
SELECT employee_id, first_name, last_name, job_id, salary
FROM employees e
WHERE salary >
    (
        SELECT AVG(salary)
        FROM employees
        WHERE job_id = e.job_id
    );
```

### Subquery Type:

- Correlated Subquery

### Execution Logic:

- Average salary calculated within each job role.

## 5.3 Departments where all employees earn more than 5,000

### Business Requirement:

- Identify departments with no low-paid employees.

### SQL:

```
SELECT department_id
FROM employees
GROUP BY department_id
HAVING MIN(salary) > 5000;
```

### Subquery Version:

```
SELECT department_id
FROM departments d
WHERE NOT EXISTS
(
    SELECT 1
    FROM employees e
    WHERE e.department_id = d.department_id
    AND e.salary <= 5000
);
```

### Subquery Type:

- Correlated EXISTS Subquery

### Execution Logic:

- Department returned only if no employee earns 5,000 or less.

## 5.4 Employees who are the only employees in their department

### Business Requirement:

- Find single-member departments.

### SQL:

```
SELECT employee_id, first_name, last_name, department_id
FROM employees e
WHERE 1 =
    (
        SELECT COUNT(*)
        FROM employees
        WHERE department_id = e.department_id
    );
```

### Subquery Type:

- Correlated Scalar Subquery

### Execution Logic:

- Count employees in employee's department; return only if count = 1.

## 6. Executive HR Analytics Challenge

### 6.1 Employees earning above company average salary

### SQL:

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary >
    (
        SELECT AVG(salary)
        FROM employees
    );
```

### Type:

- Scalar Subquery

## 6.2 Departments with more employees than Department 50

### Business Requirement:

- Compare department sizes to Department 50.

### SQL:

```
SELECT department_id, COUNT(*) employee_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) >
(
    SELECT COUNT(*)
    FROM employees
    WHERE department_id = 50
);
```

### Type:

- Scalar Subquery

## 6.3 Employees working in the department with the highest average salary

### SQL:

```
SELECT employee_id, first_name, last_name, department_id
FROM employees
WHERE department_id =
(
    SELECT department_id
    FROM
    (
        SELECT department_id,
            AVG(salary) avg_sal
        FROM employees
        GROUP BY department_id
        ORDER BY avg_sal DESC
    )
    WHERE ROWNUM = 1
);
```

### Type:

- Inline View
- Nested Subquery

### Execution Logic:

- Find department with highest average salary, then return it to its employees.

## 6.4 Employees earning the second-highest salary

**SQL:**

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary =
(
    SELECT MAX(salary)
    FROM employees
    WHERE salary <
        (
            SELECT MAX(salary)
            FROM employees
        )
);
```

**Type:**

- Nested Scalar Subquery

**Execution Logic:**

1. Find highest salary.
2. Find maximum salary below it.
3. Return employee(s) with that salary.

## 6.5 Managers whose employees' average salary exceeds 10,000

**SQL:**

```
SELECT employee_id, first_name, last_name
FROM employees m
WHERE employee_id IN
(
    SELECT manager_id
    FROM employees
    WHERE manager_id IS NOT NULL
    GROUP BY manager_id
    HAVING AVG(salary) > 10000
);
```

**Type:**

- Multi-row Subquery
- Aggregation Subquery

**Execution Logic:**

1. Group employees by manager.
2. Calculate average salary per manager.
3. Return managers whose team's average salary exceeds 10,000.